```
# Scenario: AI System for Detecting Lung Diseases from Chest X-rays
# 🚨 The Problem

# A large hospital receives thousands of chest X-rays daily.

# Radiologists are:

# Overworked

# Limited in number

# Required to make fast decisions

# Sometimes critical conditions like:

# 👉 Pneumonia
# 👉 COVID-19
# 👉 Normal lungs

# must be identified within minutes.

# Delays can cost lives.

# 💡 The Solution: Build an AI Assistant

# The hospital decides to deploy an AI model that can pre-screen X-rays and
# alert doctors.

# But there is a challenge…

# ❗ Medical datasets are usually SMALL.

# Training a deep neural network from scratch would require:

# Millions of labeled X-rays

# Massive GPU clusters

# Months of training

# Not practical.

# ⭐ Enter Transfer Learning (Your Code)

# Instead of starting from zero, engineers use:

# 👉 ResNet50 trained on ImageNet

# Although ImageNet contains everyday objects (dogs, cars, etc.),
# the early CNN layers learn universal visual patterns, like:
```

```
# ✅ Edges
# ✅ Gradients
# ✅ Textures
# ✅ Shapes

# These features are also present in medical scans.

# Import PyTorch library (main deep learning framework)
import torch

# Import torchvision models (contains pre-trained CNN architectures)
import torchvision.models as models

# Import neural network module from PyTorch
from torch import nn


# -------------------------------------------------------
# Step 1: Load a Pre-trained ResNet50 Model
# -------------------------------------------------------

# ResNet50 is a deep convolutional neural network with 50 layers.
# It is already trained on the ImageNet dataset (1.2 million images, 1000 classes).
# Using a pre-trained model allows us to reuse learned visual features
# such as edges, textures, shapes, and patterns.

model = models.resnet50(pretrained=True)


# -------------------------------------------------------
# Step 2: Freeze All Pre-trained Layers
# -------------------------------------------------------

# Transfer learning strategy:
# We freeze earlier layers so their weights do not change during training.
# These layers already learned general image features.

for param in model.parameters():
    param.requires_grad = False

# requires_grad = False means:
# - Gradients will NOT be computed
# - Weights will NOT be updated during backpropagation
# - Training becomes faster and needs less data


# -------------------------------------------------------
# Step 3: Modify the Final Classification Layer
# -------------------------------------------------------

# The original ResNet50 final layer predicts 1000 classes (ImageNet).
# Our problem only has 3 classes:
# 1. Normal
```

```
# 2. Pneumonia
# 3. COVID-19

num_classes = 3

# Replace the last fully connected layer (fc)
# model.fc.in_features gives the number of input features to the layer
# (2048 for ResNet50)

model.fc = nn.Linear(model.fc.in_features, num_classes)

# Now the model output will be 3 neurons instead of 1000.


# -------------------------------------------------------
# Step 4: Train Only the New Layer
# -------------------------------------------------------

# Since earlier layers are frozen,
# only the new fully connected layer will be trained.

trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f"Trainable params: {trainable_params}")


# -------------------------------------------------------
# Expected Output Explanation
# -------------------------------------------------------

# The output will show the number of parameters that are trainable.
# Because we froze the backbone network, only the new classification
# layer parameters will be updated during training.

# This is a typical Transfer Learning workflow used in:
# - Medical Image Analysis
# - Face Recognition
# - Object Detection
# - Small dataset problems
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, plea
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and m
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to /root/.cache/torch/hub/checkpoints/resnet50-0676ba61.pth
100%|██████████| 97.8M/97.8M [00:00<00:00, 106MB/s]
Trainable params: 6147
```

```
# Scenario: AI System for Detecting Fraud in Financial Transactions
# 🚨 The Problem
# A major bank processes millions of transactions daily.
# Fraud analysts are:
# - Overwhelmed by the sheer volume
```

```
# - Limited in number
# - Required to make instant decisions
# - Missing subtle fraud patterns hidden in massive datasets
# Critical cases like:
# 👉 Credit card fraud
# 👉 Money laundering
# 👉 Normal transactions
# must be flagged in real-time.
# Delays can cost millions and damage customer trust.

# 💡 The Solution: Build an AI Assistant
# The bank deploys an AI model that can pre-screen transactions and alert analysts.
# But here’s the challenge…
# ❗ Fraud datasets are usually imbalanced and small.
# Training a deep neural network from scratch would require:
# - Billions of labeled transactions
# - Huge compute clusters
# - Months of training
# Not practical.

# ⭐ Enter Transfer Learning (Your Code)
# Instead of starting from zero, engineers use:
# 👉 BERT (Bidirectional Encoder Representations from Transformers) trained on massive text corpora.
# Although BERT was trained on general language (Wikipedia, books, etc.), its early layers learn universal text patterns, like:
# - ✅ Word embeddings
# - ✅ Sentence structures
# - ✅ Semantic relationships
# - ✅ Contextual meaning
# These features are also present in transaction descriptions, merchant details, and customer behavior logs.
# By fine-tuning BERT on the bank’s fraud dataset, the AI can quickly learn to distinguish:
# - Suspicious vs. normal transactions
# - Fraud rings vs. genuine customer activity

# ⚡ Just like ResNet50 helps radiologists with X-rays, BERT helps fraud analysts with transaction logs —
# both leveraging transfer learning to solve problems where data is scarce but decisions are critical.

# !pip install transformers torch scikit-learn pandas

# STEP 1 — Import Libraries
import pandas as pd
import torch
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

from transformers import (
    BertTokenizer,
    BertForSequenceClassification,
    Trainer,
    TrainingArguments
)


# STEP 2 — Create Sample Dataset
# (You can replace this with real bank data)
```

```
# (You can replace this with real bank data)
data = {
    "text": [
        "Payment to Amazon for electronics",
        "Transfer to unknown account at midnight",
        "Salary credited from company",
        "Multiple transactions to foreign accounts",
        "Grocery purchase at local store",
        "Large withdrawal from ATM at night",
        "Online shopping on Flipkart",
        "Suspicious transfer to offshore account"
    ],
    "label": [0, 1, 0, 1, 0, 1, 0, 1]  # 0 = Normal, 1 = Fraud
}

df = pd.DataFrame(data)


# STEP 3 — Train-Test Split
train_texts, val_texts, train_labels, val_labels = train_test_split(
    df["text"].tolist(),
    df["label"].tolist(),
    test_size=0.2
)


# STEP 4 — Load BERT Tokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")


# STEP 5 — Tokenization
train_encodings = tokenizer(
    train_texts,
    truncation=True,
    padding=True
)

val_encodings = tokenizer(
    val_texts,
    truncation=True,
    padding=True
)


# STEP 6 — Create Dataset Class
class FraudDataset(torch.utils.data.Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        item["labels"] = torch.tensor(self.labels[idx])
        return item
```

```
    def __len__(self):
        return len(self.labels)


train_dataset = FraudDataset(train_encodings, train_labels)
val_dataset = FraudDataset(val_encodings, val_labels)


# STEP 7 — Load Pretrained BERT Model

model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# STEP 8 — Training Configuration
training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    logging_dir="./logs",
    logging_steps=10
)


# STEP 9 — Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset
)

# STEP 10 — Train Model
trainer.train()


# STEP 11 — Prediction Function
def predict(transaction_text):

    inputs = tokenizer(
        transaction_text,
        return_tensors="pt",
        truncation=True,
        padding=True
    )

    # Move inputs to same device as model
    inputs = {key: val.to(device) for key, val in inputs.items()}

    outputs = model(**inputs)
```

```
    logits = outputs.logits

    predicted_class = torch.argmax(logits).item()

    if predicted_class == 1:
        return "🚨 Fraud Detected"
    else:
        return "✅ Normal Transaction"

# STEP 12 — Test Predictions
print(predict("Transfer of $5000 to unknown offshore account"))
print(predict("Payment at local grocery store"))
```

```
Loading weights: 100%                                    199/199 [00:01<00:00, 204.46it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                        | Status     |
-------------------------------------------+------------+-
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.seq_relationship.weight                | UNEXPECTED |
cls.predictions.bias                       | UNEXPECTED |
cls.predictions.transform.dense.bias       | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |
cls.predictions.transform.dense.weight     | UNEXPECTED |
cls.seq_relationship.bias                  | UNEXPECTED |
classifier.weight                          | MISSING    |
classifier.bias                            | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGING_DIR` instead.
[6/6 00:12, Epoch 3/3]
```

Step	Training Loss

```
Writing model shards: 100%                                1/1 [00:02<00:00,  2.13s/it]
✅ Normal Transaction
✅ Normal Transaction
```

```
# mehtod 2 of the above
# ============================
# 1. Install (run once)
# ============================
# pip install transformers datasets torch scikit-learn pandas
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# ============================
# 2. Imports
# ============================
import pandas as pd
```

```
import torch
from sklearn.model_selection import train_test_split
from transformers import BertTokenizer, BertForSequenceClassification, Trainer, TrainingArguments

# ============================
# 3. Sample Dataset
# ============================
data = {
    "text": [
        "Payment to Amazon for electronics",
        "Suspicious transfer to offshore account",
        "Grocery shopping at local store",
        "Multiple rapid transactions in foreign country",
        "Salary credited from employer",
        "Unusual large withdrawal at midnight"
    ],
    "label": [0, 1, 0, 1, 0, 1]  # 0 = Normal, 1 = Fraud
}

df = pd.DataFrame(data)

# ============================
# 4. Train-Test Split
# ============================
train_texts, val_texts, train_labels, val_labels = train_test_split(
    df["text"], df["label"], test_size=0.2, random_state=42
)

# ============================
# 5. Tokenization
# ============================
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

train_encodings = tokenizer(list(train_texts), truncation=True, padding=True)
val_encodings = tokenizer(list(val_texts), truncation=True, padding=True)

# ============================
# 6. Dataset Class
# ============================
class FraudDataset(torch.utils.data.Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = list(labels)

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        item["labels"] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        return len(self.labels)

train_dataset = FraudDataset(train_encodings, train_labels)
val_dataset = FraudDataset(val_encodings, val_labels)
```

```
val_dataset = FraudDataset(val_encodings, val_labels)

# ============================
# 7. Load BERT Model
# ============================
model = BertForSequenceClassification.from_pretrained(
    'bert-base-uncased',
    num_labels=2
)

model.to(device)   # ✅ ADD THIS

# ============================
# 8. Training Arguments (FIXED)
# ============================
training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    logging_dir="./logs",
    save_strategy="epoch"
    # ❌ removed evaluation_strategy (fix for your error)
)

# ============================
# 9. Trainer
# ============================
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset
)

# ============================
# 10. Train Model
# ============================
trainer.train()

# ============================
# 11. Evaluate
# ============================
trainer.evaluate()

# ============================
# 12. Prediction Function
# ============================
def predict(text):

    model.eval()  # good practice

    inputs = tokenizer(
        text,
```

```
        return_tensors="pt",
        truncation=True,
        padding=True
    )

    # ✅ move inputs to same device
    inputs = {k: v.to(device) for k, v in inputs.items()}

    outputs = model(**inputs)

    probs = torch.nn.functional.softmax(outputs.logits, dim=1)

    return "Fraud" if torch.argmax(probs).item() == 1 else "Normal"

# ============================
# 13. Test Prediction
# ============================
print(predict("Unusual transaction in foreign country at 2AM"))
```

```
Loading weights: 100%                                    199/199 [00:00<00:00, 299.93it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                        | Status     |
-------------------------------------------+------------+-
cls.predictions.transform.dense.weight     | UNEXPECTED |
cls.predictions.transform.dense.bias       | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.bias                       | UNEXPECTED |
cls.seq_relationship.weight                | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |
cls.seq_relationship.bias                  | UNEXPECTED |
classifier.weight                          | MISSING    |
classifier.bias                            | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGING_DIR` instead.
[3/3 01:45, Epoch 3/3]

 Step  Training Loss

Writing model shards: 100%                                    1/1 [00:03<00:00,   3.28s/it]

Writing model shards: 100%                                    1/1 [00:36<00:00, 36.95s/it]

Writing model shards: 100%                                    1/1 [00:02<00:00,   2.59s/it]

[1/1 : < :]
Fraud
```

```
#  The Solution: Build an AI Assistant
# The university deploys an AI model that can pre-screen essays and alert professors about suspicious content.
# But here’s the challenge…
# ❗ Academic plagiarism datasets are usually small and domain-specific.
# Training a deep NLP model from scratch would require:
# - Millions of labeled essays
```

```
# - Huge compute resources
# - Months of training
# Not practical.

# ⭐ Enter Transfer Learning (Your Code)
# Instead of starting from zero, engineers use:
# 👉 GPT-style language models trained on massive text corpora (books, articles, Wikipedia).
# Although these corpora contain general language, the early layers learn universal text features, like:
# - ✅ Grammar patterns
# - ✅ Semantic meaning
# - ✅ Sentence structures
# - ✅ Contextual relationships
# These features are also present in student essays.
# By fine-tuning the model on a smaller plagiarism dataset, the AI can quickly learn to distinguish:
# - Original vs. copied content
# - Paraphrased vs. genuinely written text
# - Citation misuse vs. proper referencing

# ⚡ Just like ResNet50 helps radiologists with X-rays, and BERT helps fraud analysts with transactions, GPT-style models help
# educators with essays — all leveraging transfer learning to solve problems where data is scarce but accuracy is critical.


import pandas as pd
import torch
from sklearn.model_selection import train_test_split
from transformers import BertTokenizer, BertForSequenceClassification, Trainer, TrainingArguments

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

data = {
    "text": [
        "This essay is written originally by the student",
        "Copied content from Wikipedia article",
        "The student explains the concept in their own words",
        "This paragraph is directly taken from an online blog",
        "Proper citation is used for all references",
        "Content copied without citation from research paper"
    ],
    "label": [0, 1, 0, 1, 0, 1]   # 0 = Original, 1 = Plagiarized
}

df = pd.DataFrame(data)

train_texts, val_texts, train_labels, val_labels = train_test_split(
    df["text"], df["label"], test_size=0.2, random_state=42
)

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

train_encodings = tokenizer(list(train_texts), truncation=True, padding=True)
val_encodings = tokenizer(list(val_texts), truncation=True, padding=True)

class Dataset(torch.utils.data.Dataset):
```

```
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = list(labels)

    def __getitem__(self, idx):
        item = {k: torch.tensor(v[idx]) for k, v in self.encodings.items()}
        item["labels"] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        return len(self.labels)

train_dataset = Dataset(train_encodings, train_labels)
val_dataset = Dataset(val_encodings, val_labels)

model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)
model.to(device)

training_args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    logging_steps=10
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset
)

trainer.train()

def predict(text):
    model.eval()
    inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)
    inputs = {k: v.to(device) for k, v in inputs.items()}
    outputs = model(**inputs)
    probs = torch.nn.functional.softmax(outputs.logits, dim=1)
    return "Plagiarized" if torch.argmax(probs).item() == 1 else "Original"

print(predict("This paragraph is copied from an online source"))
print(predict("The student explains everything in their own words"))
```

```
Loading weights: 100%                                    199/199 [00:01<00:00, 191.13it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                       | Status     |
------------------------------------------+------------+-
cls.predictions.transform.dense.weight    | UNEXPECTED |
cls.predictions.transform.dense.bias      | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.bias                      | UNEXPECTED |
cls.seq_relationship.weight               | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias  | UNEXPECTED |
cls.seq_relationship.bias                 | UNEXPECTED |
classifier.weight                         | MISSING    |
classifier.bias                           | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
[3/3 00:49, Epoch 3/3]
```

Step	Training Loss

```
Writing model shards: 100%                                    1/1 [00:32<00:00, 32.76s/it]
Plagiarized
Original
```

```
# Install required libraries if not already installed:
# pip install transformers datasets peft accelerate

# Install required libraries if not already installed:
# pip install transformers datasets peft accelerate

from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments, Trainer
from datasets import Dataset
from peft import get_peft_model, LoraConfig

# 1. Load base model and tokenizer
model_name = "gpt2"   # you can replace with another causal LM
model = AutoModelForCausalLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Ensure pad token exists (GPT-2 doesn't have one by default)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# 2. Define LoRA configuration
lora_config = LoraConfig(
    r=8,                  # rank of low-rank matrices
    lora_alpha=32,        # scaling factor
    lora_dropout=0.1,     # dropout for regularization
    bias="none",          # no bias terms updated
    task_type="CAUSAL_LM",# language modeling task
    target_modules=["c_attn"]  # GPT-2 attention projection layers
)
```

```
# Wrap model with LoRA adapters
model = get_peft_model(model, lora_config)

# 3. Create a tiny synthetic dataset
train_texts = ["Hello world", "LoRA fine-tuning is fun", "Transformers are powerful"]
eval_texts = ["Testing evaluation", "Another sample"]

train_dataset = Dataset.from_dict({"text": train_texts})
eval_dataset = Dataset.from_dict({"text": eval_texts})

# Tokenization function with labels
def tokenize_function(examples):
    tokens = tokenizer(
        examples["text"],
        truncation=True,
        padding="max_length",
        max_length=64
    )
    tokens["labels"] = tokens["input_ids"].copy()  # labels required for loss
    return tokens

# Apply tokenization
train_dataset = train_dataset.map(tokenize_function, batched=True)
eval_dataset = eval_dataset.map(tokenize_function, batched=True)

# Set format for PyTorch
train_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])
eval_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])

# 4. Training configuration
training_args = TrainingArguments(
    output_dir="./lora-finetuned-model",   # save directory
    per_device_train_batch_size=2,         # small batch size
    gradient_accumulation_steps=2,         # accumulate gradients
    learning_rate=2e-4,                    # tuned for LoRA
    num_train_epochs=1,                    # demo run
    logging_steps=1,                       # log every step
    save_strategy="epoch",                 # save at end of epoch
    fp16=True                              # mixed precision training
)

# 5. Initialize Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset
)

# 6. Train
trainer.train()
```

```
# 7. Save only LoRA adapter weights (few MB instead of GBs!)
model.save_pretrained("./lora-weights")

print("Training complete. LoRA weights saved in ./lora-weights")
```

```
Loading weights: 100%                                    148/148 [00:01<00:00, 106.85it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                  | Status     |  |
---------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Map: 100%                                    3/3 [00:00<00:00, 27.36 examples/s]

Map: 100%                                    2/2 [00:00<00:00, 22.21 examples/s]
`loss_type=None` was set in the config but it is unrecognized. Using the default loss: `ForCausalLMLoss`.
[1/1 00:00, Epoch 1/1]
```

Step	Training Loss
1	8.129440

```
Training complete. LoRA weights saved in ./lora-weights
```

```
# Big Idea Behind This Code

# Instead of shipping a 500MB–10GB model every time…

# You ship:

# 👉 A tiny adapter (few MB)

# Then dynamically attach it to the base model.

# This enables:

# ✅ Modular AI systems
# ✅ Fast deployment
# ✅ Cheap storage
# ✅ Multi-domain adapters

# This pattern is exploding across the industry.

# 🚀 Real-World Scenario (VERY Powerful)
# 🛒 Scenario: E-Commerce Company Builds Multiple AI Assistants

# An e-commerce giant wants different AI behaviors:

# Sentiment analyzer

# Product recommender

# Customer support bot
```

```
# Return-policy assistant

# But retraining separate LLMs would cost millions.

# So engineers do something smart:

# 👉 One Base Model
# 👉 Multiple LoRA Adapters


# Install required libraries if not already installed:
# pip install transformers datasets peft accelerate

from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments, Trainer
from datasets import Dataset
from peft import get_peft_model, LoraConfig, PeftModel

# 1. Load base model and tokenizer
model_name = "gpt2"   # you can replace with another causal LM
model = AutoModelForCausalLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Ensure pad token exists
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# 2. Define LoRA configuration
lora_config = LoraConfig(
    r=8,
    lora_alpha=32,
    lora_dropout=0.1,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=["c_attn"]  # GPT-2 uses 'c_attn' for QKV projection
)

# Wrap model with LoRA
model = get_peft_model(model, lora_config)

# 3. Create a tiny synthetic dataset
train_texts = ["Hello world", "LoRA fine-tuning is fun", "Transformers are powerful"]
eval_texts = ["Testing evaluation", "Another sample"]

train_dataset = Dataset.from_dict({"text": train_texts})
eval_dataset = Dataset.from_dict({"text": eval_texts})

# Tokenization function with labels
def tokenize_function(examples):
    tokens = tokenizer(
        examples["text"],
        truncation=True,
        padding="max_length",
        max_length=64
```

```
        max_length=64
    )
    tokens["labels"] = tokens["input_ids"].copy()  # labels required for loss
    return tokens

train_dataset = train_dataset.map(tokenize_function, batched=True)
eval_dataset = eval_dataset.map(tokenize_function, batched=True)

# Set format for PyTorch
train_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])
eval_dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "labels"])

# 4. Training configuration
training_args = TrainingArguments(
    output_dir="./lora-finetuned-model",
    per_device_train_batch_size=2,
    gradient_accumulation_steps=2,
    learning_rate=2e-4,
    num_train_epochs=1,   # keep small for demo
    logging_steps=1,
    save_strategy="epoch",
    fp16=True
)

# 5. Initialize Trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset
)

# 6. Train
trainer.train()

# 7. Save only LoRA adapter weights (few MB instead of GBs!)
model.save_pretrained("lora-sentiment")  # folder will contain adapter_config.json

print("Training complete. LoRA weights saved in ./lora-sentiment")

# 8. Reload base model and attach adapter for inference
base_model = AutoModelForCausalLM.from_pretrained("gpt2")
sentiment_model = PeftModel.from_pretrained(base_model, "lora-sentiment")

# Example inference
input_text = "I love using transformers and LoRA adapters!"
input_ids = tokenizer(input_text, return_tensors="pt").input_ids
output_ids = sentiment_model.generate(input_ids, max_length=50)
print("Output:", tokenizer.decode(output_ids[0], skip_special_tokens=True))
```

```
config.json: 100%                                                    665/665 [00:00<00:00, 36.1kB/s]

model.safetensors: 100%                                              548M/548M [00:12<00:00, 48.7MB/s]

Loading weights: 100%                                                148/148 [00:00<00:00, 561.14it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                   | Status     |  |
----------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
generation_config.json: 100%                                         124/124 [00:00<00:00, 7.46kB/s]

tokenizer_config.json: 100%                                          26.0/26.0 [00:00<00:00, 3.08kB/s]

vocab.json:        1.04M/? [00:00<00:00, 4.96MB/s]

merges.txt:        456k/? [00:00<00:00, 2.49MB/s]

tokenizer.json:        1.36M/? [00:00<00:00, 4.81MB/s]
/usr/local/lib/python3.12/dist-packages/peft/tuners/lora/layer.py:2285: UserWarning: fan_in_fan_out is set to False but the target module is `Conv1D`. Setting fan_in_fan_out to
  warnings.warn(
Map: 100%                                                 3/3 [00:00<00:00, 50.15 examples/s]

Map: 100%                                                 2/2 [00:00<00:00, 124.97 examples/s]
`loss_type=None` was set in the config but it is unrecognized. Using the default loss: `ForCausalLMLoss`.
[1/1 00:00, Epoch 1/1]
```

Step	Training Loss
1	8.163726

```
Training complete. LoRA weights saved in ./lora-sentiment
Loading weights: 100%                                                148/148 [00:00<00:00, 592.47it/s, Materializing param=transformer.wte.weight]
GPT2LMHeadModel LOAD REPORT from: gpt2
Key                   | Status     |  |
----------------------+------------+--+-
h.{0...11}.attn.bias | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
The attention mask and the pad token id were not set. As a consequence, you may observe unexpected behavior. Please pass your input's `attention_mask` to obtain reliable results
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, you may observe unexpected behavior. Please pass your i
Output: I love using transformers and LoRA adapters!

I love using transformers and LoRA adapters! I love using transformers and LoRA adapters! I love using transformers and LoRA adapters! I love using transformers and LoRA
```

```
# LoRA Application
# Scenario: Fraud Detection in Customer Support Chats
# - Problem: Fraudsters often contact bank support pretending to be customers, trying to reset passwords or gain access.
# - Challenge: Analysts can’t manually read millions of chat transcripts.
# - Solution: Use LoRA fine-tuning on a pretrained language model (like distilbert-base-uncased) to classify chats as:
```

```
# - 0 → Normal inquiry
# - 1 → Fraud attempt
# - 2 → Suspicious but unclear
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification, TrainingArguments, Trainer
from datasets import Dataset
from peft import get_peft_model, LoraConfig, TaskType, PeftModel

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model_name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=3
)

model.to(device)

lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0.1,
    bias="none",
    task_type=TaskType.SEQ_CLS,
    target_modules=["q_lin", "v_lin"]  # DistilBERT attention layers
)

model = get_peft_model(model, lora_config)

train_data = {
    "text": [
        "I forgot my password can you help reset it",
        "Please change my account email urgently",
        "Hello I want to check my balance",
        "Transfer all funds to this new account now",
        "Can you guide me about loan options",
        "I lost access please disable security quickly"
    ],
    "label": [0, 2, 0, 1, 0, 2]  # 0 normal, 1 fraud, 2 suspicious
}

dataset = Dataset.from_dict(train_data)

def tokenize(example):
    return tokenizer(example["text"], truncation=True, padding="max_length", max_length=64)

dataset = dataset.map(tokenize)
dataset.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])

model.train()
```

```
training_args = TrainingArguments(
    output_dir="./lora-fraud",
    per_device_train_batch_size=2,
    num_train_epochs=3,
    logging_steps=1
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset
)

trainer.train()

model.save_pretrained("lora-fraud")

base_model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=3
)

model = PeftModel.from_pretrained(base_model, "lora-fraud")
model.to(device)
model.eval()

def predict(text):
    inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)
    inputs = {k: v.to(device) for k, v in inputs.items()}

    outputs = model(**inputs)
    probs = torch.softmax(outputs.logits, dim=1)
    pred = torch.argmax(probs).item()

    if pred == 0:
        return "Normal Inquiry"
    elif pred == 1:
        return "Fraud Attempt 🚨"
    else:
        return "Suspicious ⚠️"

print(predict("Please reset my password immediately"))
print(predict("I want to know my account balance"))
```